

CS1675/2075: Introduction to Machine Learning

Week 10: Categorical Input Features

Fall 2024

We have spent a lot of time with continuous inputs...

- What would happen if we have a discrete or categorical input?
- For example, what if an input is binary? And can take on two possible states:

$$x = \{A, B\}$$

We have spent a lot of time with continuous inputs...

- What would happen if we have a discrete or categorical input?
- Or, what if an input can take on 4 possible states?

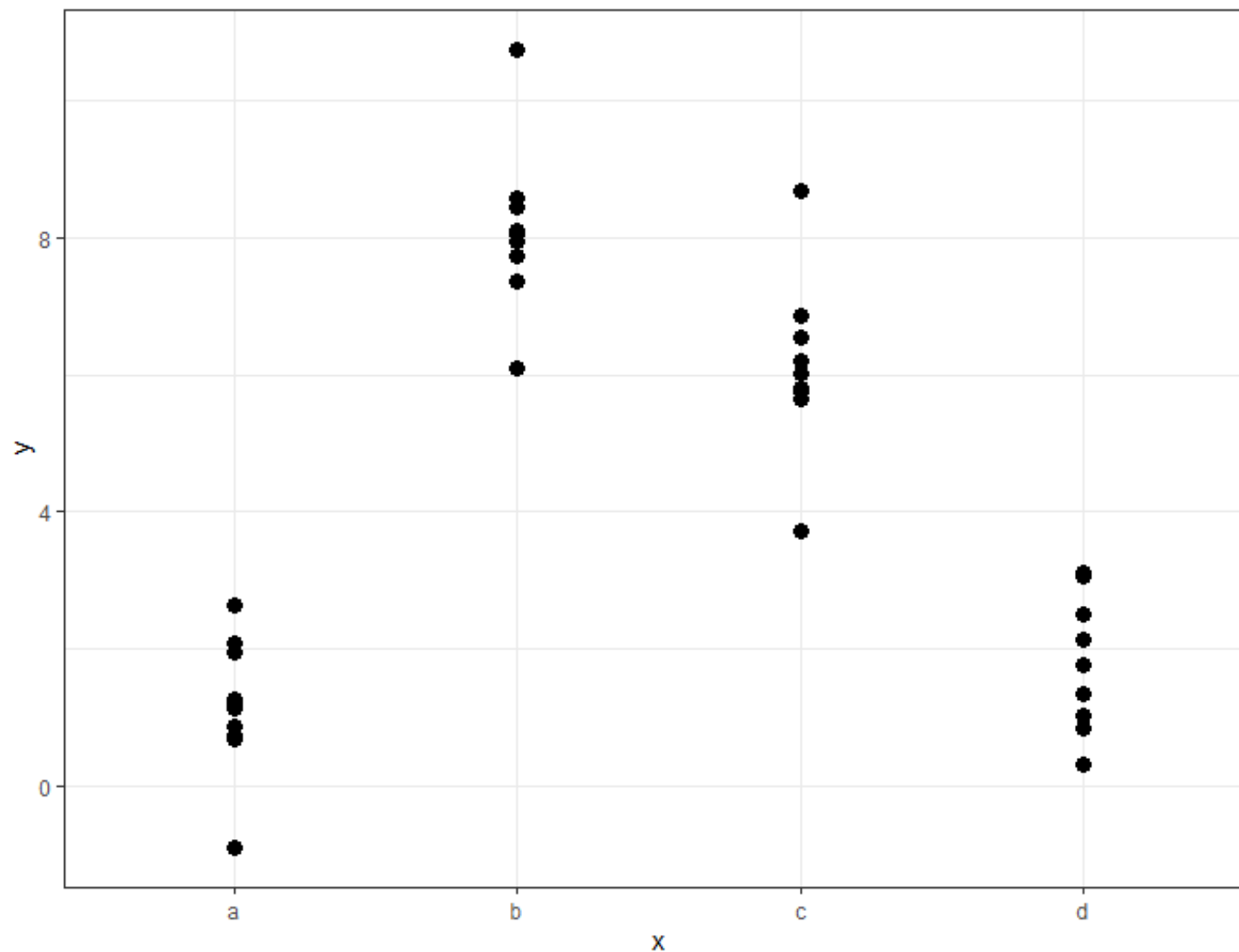
$$x = \{a, b, c, d\}$$

How would we handle this input? We can't multiply a slope by a character string.

The R for Data Science book has a nice example for this exact situation

- Consider a data set of a single categorical input, x , which has 4 levels, a, b, c and d.
- The response, y , depends on the input.
- This dataset is named `sim2` in the `modelr` package.

The `sim2` data from the `modelr` package



Fit a linear model with `lm ( )` and see what happens

```
> ### fit the linear model  
> mod_02 <- lm(y ~ x, data = sim2)  
> mod_02 %>% summary()
```

Fit a linear model with `lm ( )` and see what happens

```
> ### fit the linear model  
> mod_02 <- lm(y ~ x, data = sim2)  
> mod_02 %>% summary()
```

```
call:  
lm(formula = y ~ x, data = sim2)
```

```
Residuals:
```

	Min	1Q	Median	3Q	Max
	-2.40131	-0.43996	-0.05776	0.49066	2.63938

```
Coefficients:
```

	Estimate	Std. Error	t value	Pr(> t)	
(Intercept)	1.1522	0.3475	3.316	0.00209	**
xb	6.9639	0.4914	14.171	2.68e-16	***
xc	4.9750	0.4914	10.124	4.47e-12	***
xd	0.7588	0.4914	1.544	0.13131	

```
---
```

```
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
Residual standard error: 1.099 on 36 degrees of freedom
```

```
Multiple R-squared:  0.8852,    Adjusted R-squared:  0.8756
```

```
F-statistic: 92.52 on 3 and 36 DF,  p-value: < 2.2e-16
```

Fit a linear model with `lm ( )` and see what happens

```
> ### fit the linear model  
> mod_02 <- lm(y ~ x, data = sim2)  
> mod_02 %>% summary()
```

```
call:  
lm(formula = y ~ x, data = sim2)
```

```
Residuals:
```

Min	1Q	Median	3Q	Max
-2.40131	-0.43996	-0.05776	0.49066	2.63938

```
Coefficients:
```

	Estimate	Std. Error	t value	Pr(> t)	
(Intercept)	1.1522	0.3475	3.316	0.00209	**
xb	6.9639	0.4914	14.171	2.68e-16	***
xc	4.9750	0.4914	10.124	4.47e-12	***
xd	0.7588	0.4914	1.544	0.13131	

```
---  
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
Residual standard error: 1.099 on 36 degrees of freedom  
Multiple R-squared:  0.8852,    Adjusted R-squared:  0.8756  
F-statistic: 92.52 on 3 and 36 DF,  p-value: < 2.2e-16
```

There are 3
coefficients
plus the
intercept?!

How did we go from a single input to 3 coefficients?

How did we go from a single input to 3 coefficients?

- **DUMMY VARIABLES**

- Dummy variables are used to represent a categorical variable in such a way that we can multiply a numeric variable by a slope.
- Dummy variables expand or augment the feature space.

Let's look at the design matrix to see what's going on.

```
> ### show the design matrix
> Xmat_02 <- model.matrix(y ~ x, data = sim2)
> head(Xmat_02)
  (Intercept) xb xc xd
1           1  0  0  0
2           1  0  0  0
3           1  0  0  0
4           1  0  0  0
5           1  0  0  0
6           1  0  0  0
> head(sim2)
# A tibble: 6 x 2
  x         y
  <chr> <dbl>
1 a     1.94
2 a     1.18
3 a     1.24
4 a     2.62
5 a     1.11
6 a     0.866
```

Let's look at the design matrix to see what's going on.

```
> ### show the design matrix
> Xmat_02 <- model.matrix(y ~ x, data = sim2) Create the design matrix
> head(Xmat_02)
  (Intercept) xb xc xd
1           1  0  0  0
2           1  0  0  0
3           1  0  0  0
4           1  0  0  0
5           1  0  0  0
6           1  0  0  0
> head(sim2)
# A tibble: 6 x 2
  x         y
  <chr> <dbl>
1 a     1.94
2 a     1.18
3 a     1.24
4 a     2.62
5 a     1.11
6 a     0.866
```

Let's look at the design matrix to see what's going on.

```
> ### show the design matrix  
> Xmat_02 <- model.matrix(y ~ x, data = sim2)  
> head(Xmat_02)
```

	(Intercept)	xb	xc	xd
1	1	0	0	0
2	1	0	0	0
3	1	0	0	0
4	1	0	0	0
5	1	0	0	0
6	1	0	0	0

Print out the first 6 rows of the design matrix. There's the intercept column and 3 columns xb, xc, and xd.

All three columns are equal to zero.

```
> head(sim2)  
# A tibble: 6 x 2  
  x      y  
  <chr> <dbl>  
1 a     1.94  
2 a     1.18  
3 a     1.24  
4 a     2.62  
5 a     1.11  
6 a     0.866
```

Let's look at the design matrix to see what's going on.

```
> ### show the design matrix  
> Xmat_02 <- model.matrix(y ~ x, data = sim2)  
> head(Xmat_02)
```

	(Intercept)	xb	xc	xd
1	1	0	0	0
2	1	0	0	0
3	1	0	0	0
4	1	0	0	0
5	1	0	0	0
6	1	0	0	0

```
> head(sim2)
```

```
# A tibble: 6 x 2
```

	x	y
	<chr>	<dbl>
1	a	1.94
2	a	1.18
3	a	1.24
4	a	2.62
5	a	1.11
6	a	0.866

Compare to the first 6 rows of the original data set.

The input x equals a for all 6 observations.

Let's look at the design matrix to see what's going on.

```
> ### show the design matrix
> Xmat_02 <- model.matrix(y ~ x, data = sim2)
> head(Xmat_02)
```

```
(Intercept) xb xc xd
1           1  0  0  0
2           1  0  0  0
3           1  0  0  0
4           1  0  0  0
5           1  0  0  0
6           1  0  0  0
```

```
> head(sim2)
# A tibble: 6 x 2
```

```
  x      y
<chr> <dbl>
1 a     1.94
2 a     1.18
3 a     1.24
4 a     2.62
5 a     1.11
6 a     0.866
```

The dummy variables are structured as:

xb - does x equal b ? xb=1 means YES!

xc - does x equal c ? xc=1 means YES!

xd - does x equal d ? xd=1 means YES!

If all 3 dummy variables are 0 then x equals?

Let's look at the design matrix to see what's going on.

```
> ### show the design matrix
> Xmat_02 <- model.matrix(y ~ x, data = sim2)
> head(Xmat_02)
  (Intercept) xb xc xd
1           1  0  0  0
2           1  0  0  0
3           1  0  0  0
4           1  0  0  0
5           1  0  0  0
6           1  0  0  0
> head(sim2)
# A tibble: 6 x 2
  x         y
<chr> <dbl>
1 a     1.94
2 a     1.18
3 a     1.24
4 a     2.62
5 a     1.11
6 a     0.866
```

The dummy variables are structured as:
xb - does x equal b ? xb=1 means YES!
xc - does x equal c ? xc=1 means YES!
xd - does x equal d ? xd=1 means YES!

If all 3 are 0 then x equals? x=a!

When $x == \text{"b"}$, the dummy variable $xb=1$!

```
> ### look at when x == "b"
> Xmat_02[8:12, ]
      (Intercept) xb xc xd
8              1  0  0  0
9              1  0  0  0
10             1  0  0  0
11             1  1  0  0
12             1  1  0  0
> sim2 %>% slice(8:12)
# A tibble: 5 x 2
  x         y
  <chr> <dbl>
1 a     0.721
2 a     0.687
3 a     2.07
4 b     8.07
5 b     7.36
```

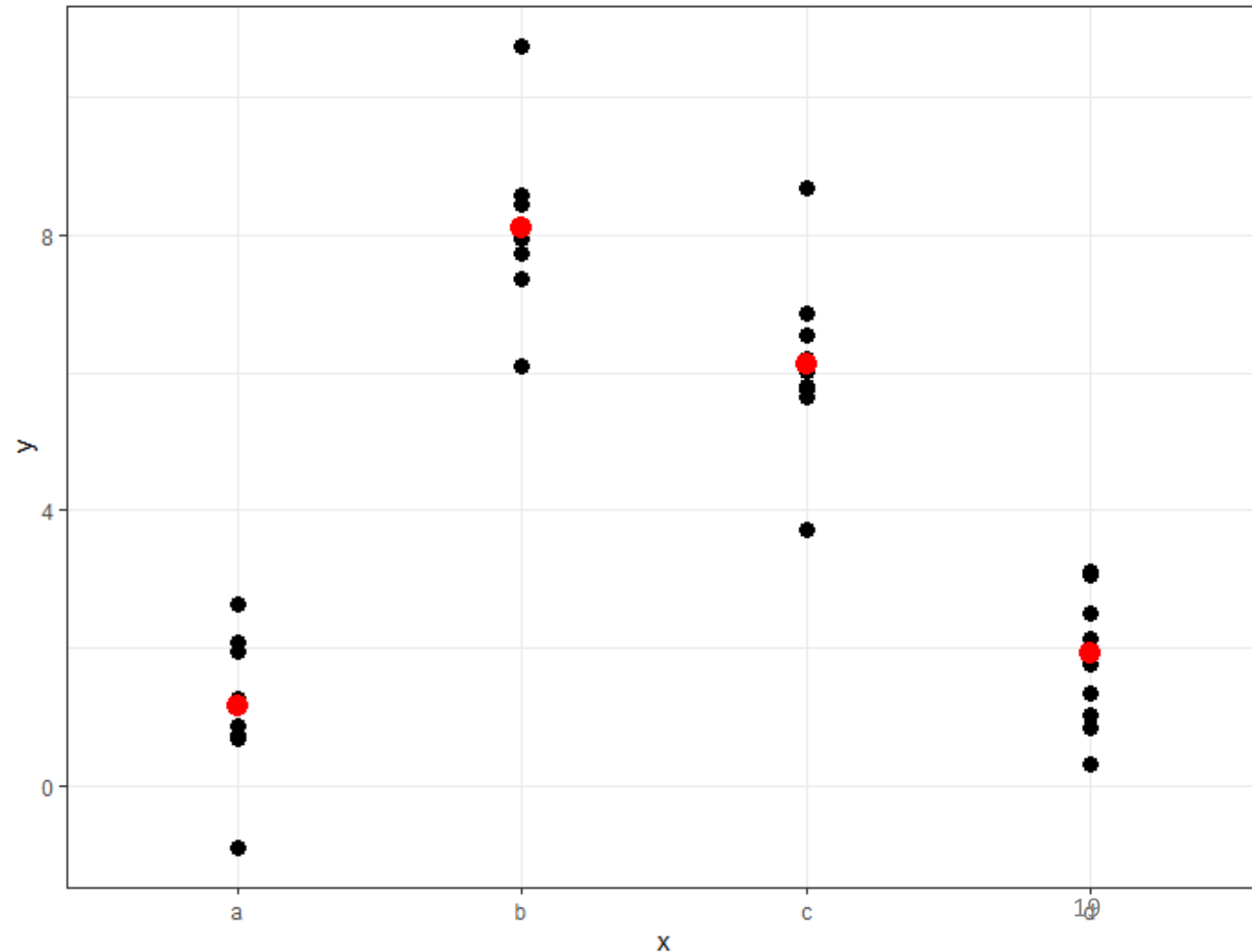
When $x == \text{"b"}$, the dummy variable $xb = 1$!

```
> ### look at when x == "b"
> Xmat_02[8:12, ]
      (Intercept) xb xc xd
8              1  0  0  0
9              1  0  0  0
10             1  0  0  0
11             1  1  0  0
12             1  1  0  0
> sim2 %>% slice(8:12)
# A tibble: 5 x 2
  x      y
  <chr> <dbl>
1 a    0.721
2 a    0.687
3 a    2.07
4 b    8.07
5 b    7.36
```

Predicting the training set reveals that the model is predicting the AVERAGE response associated with each level!

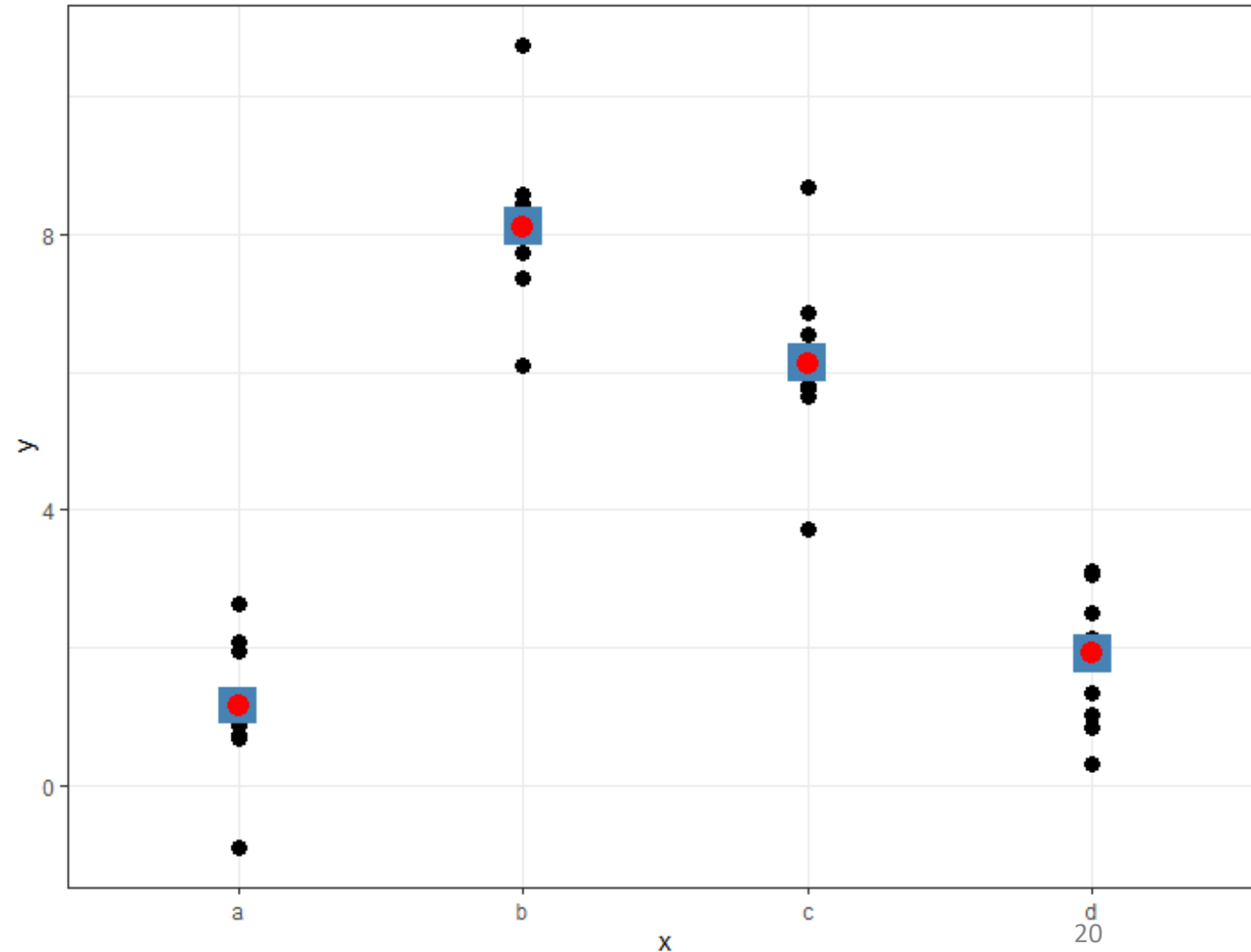
```
### make predictions on the training set
pred_train_02 <- predict(mod_02, sim2)
```

```
sim2 %>%
  mutate(ypred = pred_train_02) %>%
  ggplot(mapping = aes(x = x)) +
  geom_point(mapping = aes(y = y),
                size = 3) +
  geom_point(mapping = aes(y = ypred),
                color = "red", size = 4) +
  theme_bw()
```



Double check by comparing the predictions with the response averages associated with each level

```
### double check by calculating the average response
sim2 %>%
  mutate(ypred = pred_train_02) %>%
  ggplot(mapping = aes(x = x)) +
  geom_point(mapping = aes(y = y),
    size = 3) +
  stat_summary(fun.y = "mean",
    geom = "point",
    mapping = aes(y = y),
    size = 7, shape = 15,
    color = "steelblue") +
  geom_point(mapping = aes(y = ypred),
    color = "red", size = 4) +
  theme_bw()
```



What do the dummy variable slopes correspond to?

```
> coef(mod_02)
(Intercept)          xb          xc          xd
  1.1521664    6.9638728    4.9750241    0.7588142
> ### compare to the grouped averages
> sim2 %>%
+   group_by(x) %>%
+   summarise(num_rows = n(),
+             y_avg = mean(y)) %>%
+   ungroup()
# A tibble: 4 x 3
  x      num_rows y_avg
<chr>    <int> <dbl>
1 a         10  1.15
2 b         10  8.12
3 c         10  6.13
4 d         10  1.91
```

What do the dummy variable slopes correspond to?

```
> coef(mod_02)
(Intercept)          xb          xc          xd
  1.1521664    6.9638728    4.9750241    0.7588142
> ### compare to the grouped averages
> sim2 %>%
+   group_by(x) %>%
+   summarise(num_rows = n(),
+             y_avg = mean(y)) %>%
+   ungroup()
# A tibble: 4 x 3
  x      num_rows y_avg
<chr>    <int> <dbl>
1 a          10  1.15
2 b          10  8.12
3 c          10  6.13
4 d          10  1.91
```

The intercept equals the average response associated with the $x=a$ level!

The intercept is therefore the average response at the REFERENCE level!

What do the dummy variable slopes correspond to?

```
> coef(mod_02)
(Intercept)      xb      xc      xd
  1.1521664    6.9638728    4.9750241    0.7588142
> ### compare to the grouped averages
> sim2 %>%
+   group_by(x) %>%
+   summarise(num_rows = n(),
+             y_avg = mean(y)) %>%
+   ungroup()
# A tibble: 4 x 3
  x      num_rows y_avg
<chr>    <int> <dbl>
1 a         10  1.15
2 b         10  8.12
3 c         10  6.13
4 d         10  1.91
```

The average response associated with $x=b$ corresponds to the SUM of the INTERCEPT and the DUMMY coefficient acting on x_b .

What do the dummy variable slopes correspond to?

```
> coef(mod 02)
(Intercept)          xb          xc          xd
  1.1521664    6.9638728    4.9750241    0.7588142
> ### compare to the grouped averages
> sim2 %>%
+   group_by(x) %>%
+   summarise(num_rows = n(),
+             y_avg = mean(y)) %>%
+   ungroup()
# A tibble: 4 x 3
  x      num_rows y_avg
<chr>    <int> <dbl>
1 a         10  1.15
2 b         10  8.12
3 c         10  6.13
4 d         10  1.91
```

The same holds for x_c .

What do the dummy variable slopes correspond to?

```
> coef(mod_02)
(Intercept)          xb          xc          xd
  1.1521664    6.9638728    4.9750241    0.7588142
> ### compare to the grouped averages
> sim2 %>%
+   group_by(x) %>%
+   summarise(num_rows = n(),
+             y_avg = mean(y)) %>%
+   ungroup()
# A tibble: 4 x 3
  x      num_rows y_avg
<chr>    <int> <dbl>
1 a         10  1.15
2 b         10  8.12
3 c         10  6.13
4 d         10  1.91
```

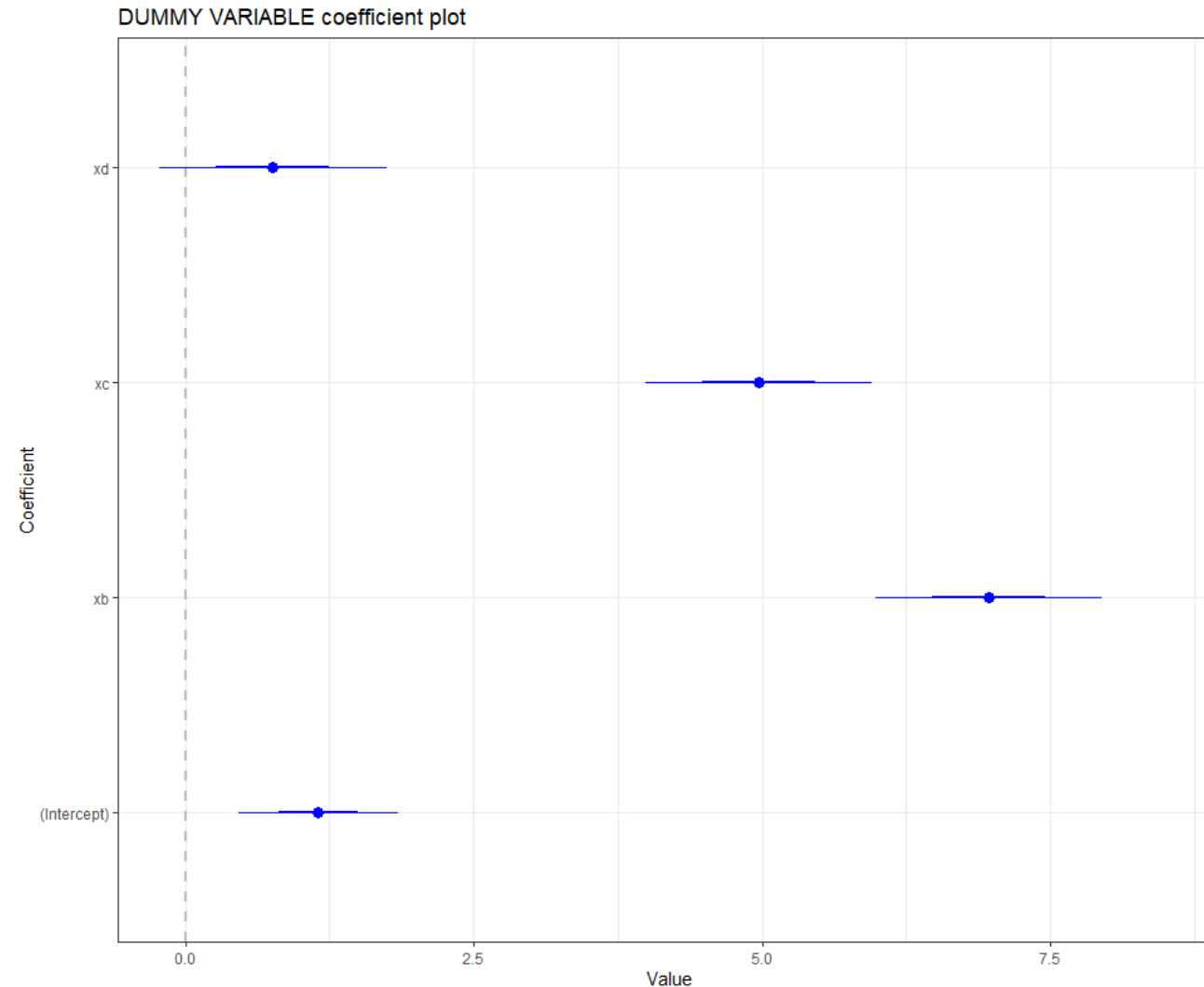
And for `xd`.

What do the dummy variable slopes correspond to?

```
> coef(mod_02)
(Intercept)          xb          xc          xd
  1.1521664    6.9638728    4.9750241    0.7588142
> ### compare to the grouped averages
> sim2 %>%
+   group_by(x) %>%
+   summarise(num_rows = n(),
+             y_avg = mean(y)) %>%
+   ungroup()
# A tibble: 4 x 3
  x      num_rows y_avg
<chr>    <int> <dbl>
1 a         10  1.15
2 b         10  8.12
3 c         10  6.13
4 d         10  1.91
```

The dummy variable coefficients are therefore the effect of the levels **RELATIVE** to the reference level.

Confidence intervals allow us to determine if the relative effect is significant



To remove the intercept and thus include all levels, we can specify -1 in the formula interface

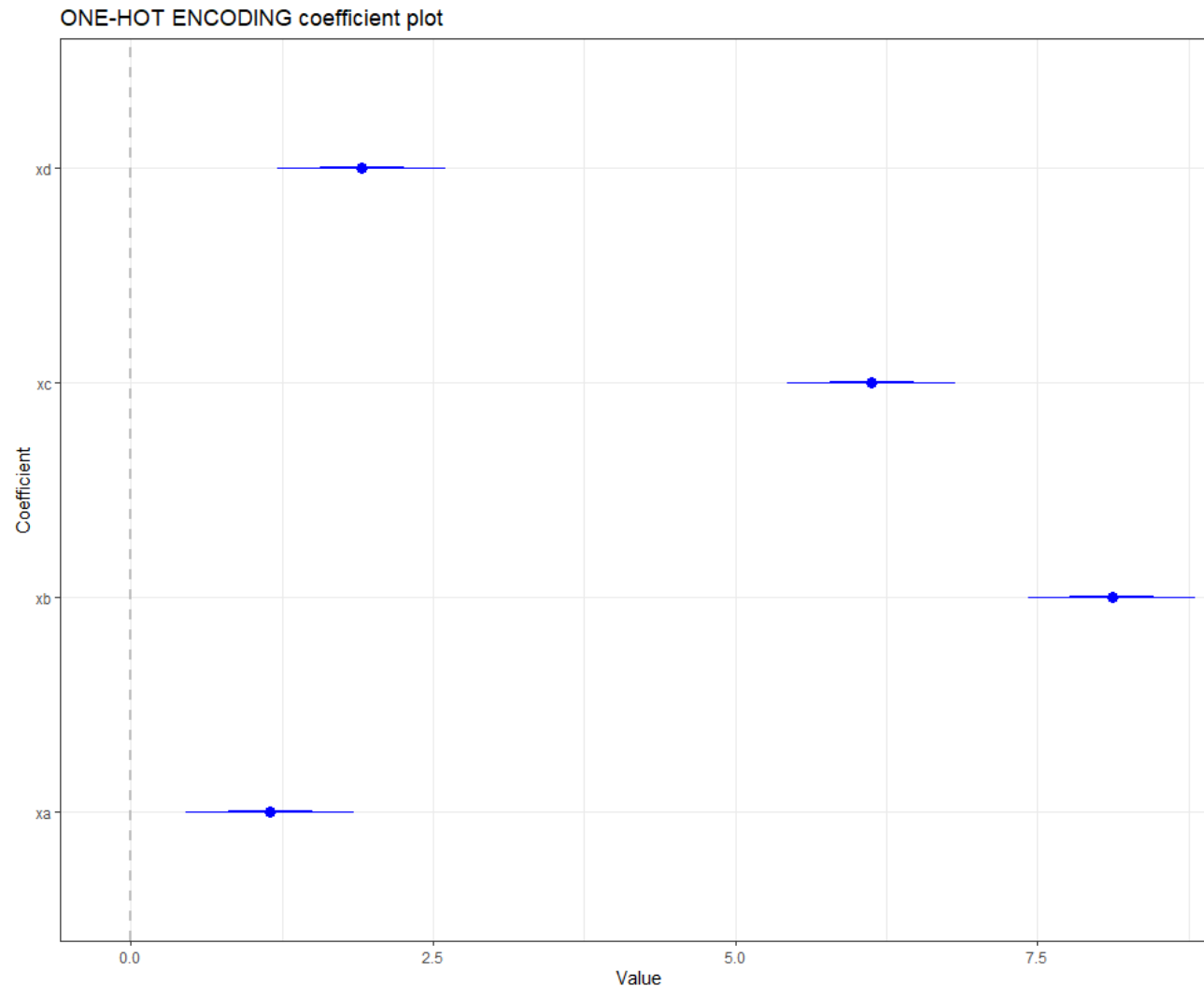
```
> model.matrix(y ~ x - 1, data = sim2) %>% head()
  xa xb xc xd
1  1  0  0  0
2  1  0  0  0
3  1  0  0  0
4  1  0  0  0
5  1  0  0  0
6  1  0  0  0
```

This is known as ONE-HOT ENCODING

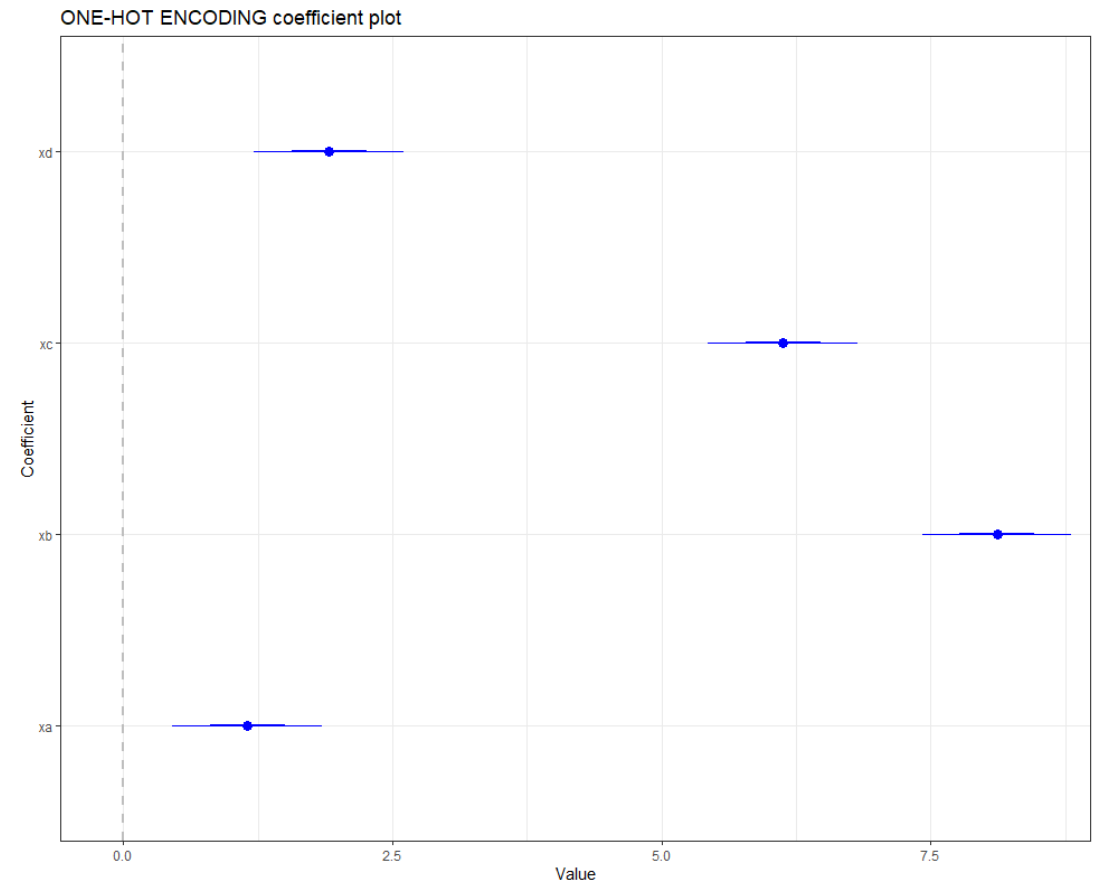
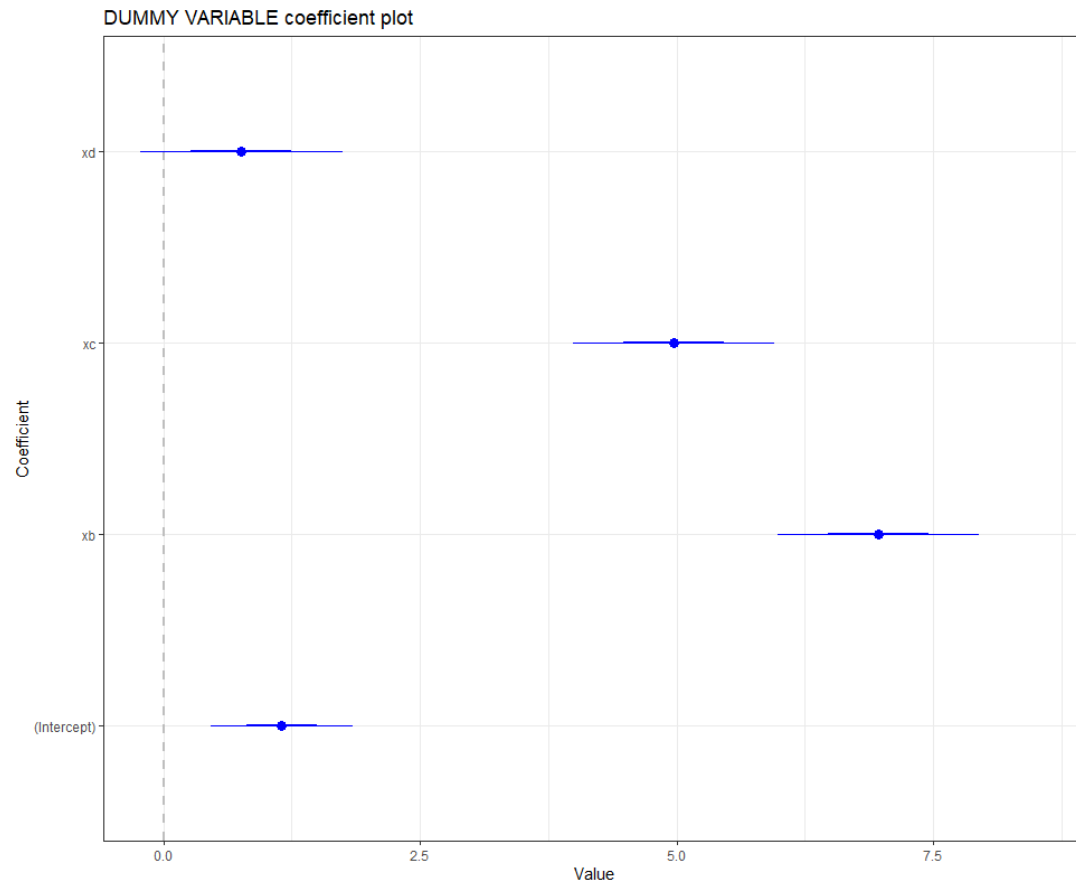
Coefficient estimates associated with each level of the categorical input now correspond to the response average!

```
> ### compare coefficient estimates associated with  
> ### the one-hot encoding with the group average values  
> sim2 %>%  
+   group_by(x) %>%  
+   summarise(y_avg = mean(y)) %>%  
+   mutate(one_hot_coef = as.vector( coef(mod_no_int) ))  
`summarise()` ungrouping output (override with `.groups` argument)  
# A tibble: 4 x 3  
  x      y_avg one_hot_coef  
  <chr> <dbl>      <dbl>  
1 a      1.15      1.15  
2 b      8.12      8.12  
3 c      6.13      6.13  
4 d      1.91      1.91
```

Visually, we need to consider if the confidence intervals overlap to determine if one level (category) has a significantly different average than another level (category)



Dummy variable approach provides a more streamlined comparison of the relative effect



IMPORTANT: If you use one-hot encoding with categorical inputs you **CANNOT** estimate the intercept!

- The intercept would not be necessary, it would be a redundant feature.
- Most modeling functions do not throw an error if you do this by mistake.
- The intercept will correspond to NA or 0 depending on the function.

In summary, to account for categorical variables we have to increase the effective number of parameters!

- In the `sim2` example, we have one input, but we must learn 4 β -parameters!

Categorical inputs and continuous inputs

- If we have a MIXTURE of input types, the concepts still hold.
- The only difference is that the INTERCEPT (reference level behavior) represents the response average when the continuous inputs equal zero.

Consider a case with one categorical, x_1 , and one continuous input, x_2 .

- We will use an additive linear relationship: $y \sim x_1 + x_2$
- Let's assume the categorical input has 4 levels, A, B, C, and D, just as in the previous example.
- The mean trend expression including the DUMMY VARIABLES is:

$$\mu_n = \beta_0 + \beta_{1B}x_{1B,n} + \beta_{1C}x_{1C,n} + \beta_{1D}x_{1D,n} + \beta_2x_{2,n}$$

- The DUMMY VARIABLES $x_{1B,n}$, $x_{1C,n}$, and $x_{1D,n}$ are equal to 0 or 1 to denote the level (category) of $x_{1,n}$.

For an observation where $x_{1,n} = A$ the DUMMY VARIABLES are all equal to ZERO!

- Original mean trend with all features:

$$\mu_n = \beta_0 + \beta_{1B}x_{1B,n} + \beta_{1C}x_{1C,n} + \beta_{1D}x_{1D,n} + \beta_2x_{2,n}$$

- When $x_{1,n} = A$ the mean trend simplifies to:

$$\mu_n = \beta_0 + \beta_2x_{2,n}$$

- The intercept, β_0 , is the average response value at the REFERENCE level when $x_2 = 0$.

For an observation where $x_{1,n} = B$ two of DUMMY VARIABLES equal ZERO!

- Original mean trend with all features:

$$\mu_n = \beta_0 + \beta_{1B}x_{1B,n} + \beta_{1C}x_{1C,n} + \beta_{1D}x_{1D,n} + \beta_2x_{2,n}$$

- When $x_{1,n} = B$ the mean trend simplifies to:

$$\mu_n = \beta_0 + \beta_{1B}x_{1B,n} + \beta_2x_{2,n}$$

For an observation where $x_{1,n} = B$ two of DUMMY VARIABLES equal ZERO!

- When $x_{1,n} = B$ the mean trend simplifies to:

$$\mu_n = \beta_0 + \beta_{1B}x_{1B,n} + \beta_2x_{2,n}$$

- The dummy variable is $x_{1B,n} = 1$, thus the above expression simplifies to:

$$\mu_n = \beta_0 + \beta_{1B} + \beta_2x_{2,n}$$

For an observation where $x_{1,n} = B$ two of DUMMY VARIABLES equal ZERO!

- When $x_{1,n} = B$ the mean trend simplifies to:

$$\mu_n = \beta_0 + \beta_{1B}x_{1B,n} + \beta_2x_{2,n}$$

- The dummy variable $x_{1B,n} = 1$, thus the above expression simplifies to:

$$\mu_n = \boxed{\beta_0 + \beta_{1B}} + \beta_2x_{2,n}$$

The sum of β_0 and β_{1B} is the effective intercept associated with the $x_1 = B$ level!
The sum is therefore the average response when $x_2 = 0$ and $x_1 = B$!

For an observation where $x_{1,n} = B$ two of DUMMY VARIABLES equal ZERO!

- When $x_{1,n} = B$ the mean trend simplifies to:

$$\mu_n = \beta_0 + \beta_{1B}x_{1B,n} + \beta_2x_{2,n}$$

- The dummy variable $x_{1B,n} = 1$, thus the above expression simplifies to:

$$\mu_n = \boxed{\beta_0 + \beta_{1B}} + \beta_2x_{2,n}$$

The DUMMY VARIABLE coefficients still represent RELATIVE effects from the reference level!

Can be useful to consider interactions between categorical inputs and continuous inputs

- The interaction represents that the slope of the continuous input **DEPENDS** on the categorical level!
- The continuous relationship to the response therefore **VARIES** across categorical groups!

More information:

- Categorical variables section in R4DS:
 - <https://r4ds.had.co.nz/model-basics.html#categorical-variables>
- Section 3.6 HOML discusses dummy variables vs one-hot-encoding, as well as lumping of rare category levels
 - <https://bradleyboehmke.github.io/HOML/engineering.html#categorical-feature-engineering>